

گزارش پروژه تبدیل زبان Hash به Promela

فاطمه باقریان – امیرحسین پیرنژاد

تیر ماه ۱۴۰۵

فهرست مطالب

۳	فاز اول	
۳	۱.۰ مقدمه	۳
۳	۲.۰ ارتباط با نظریه اتوماتا و زبان‌ها	۳
۳	۳.۰ زیرمجموعه زبان Hash پشتیبانی شده	۳
۳	۴.۰ محدودیت‌های پیاده‌سازی	۳
۴	۵.۰ نحوه عملکرد کل سیستم	۴
۵	۱ توضیح کلاس Main	۵
۵	۱.۱ خواندن ورودی	۵
۵	۲.۱ تولید Lexer و Parser	۵
۵	۳.۱ بررسی خطاهای نحوی	۵
۶	۴.۱ بررسی معنایی	۶
۶	۵.۱ تولید کد Promela	۶
۶	۶.۱ ذخیره خروجی	۶
۷	۲ چرا از Visitor استفاده شده است؟	۷
۷	۱.۲ مقایسه Visitor و Listener	۷
۷	۲.۲ دلایل اصلی انتخاب Visitor	۷
۸	۳.۲ نمونه‌های عملی از مزایای Visitor	۸
۹	۳ طراحی (HashToPromela) Translator	۹
۹	۱.۳ ساختار کلی کلاس	۹
۹	۲.۳ متد visitProgram - نقطه ورود اصلی	۹
۱۰	۳.۳ متد visitVarDecl - ترجمه تعریف متغیرها	۱۰
۱۰	۴.۳ ترجمه عبارات (Expressions) در HashToPromela	۱۰
۱۰	۱.۴.۳ روش کلی ترجمه	۱۰
۱۱	۲.۴.۳ سلسله‌مراتب ترجمه عبارات	۱۱
۱۱	۳.۴.۳ نحوه اتصال متدها	۱۱
۱۱	۴.۴.۳ عملگرهای پشتیبانی‌شده	۱۱
۱۱	۵.۴.۳ مدیریت عملگرهای خاص	۱۱
۱۲	۶.۴.۳ مزایای روش Visitor	۱۲
۱۲	۵.۳ متد visitAssignment - ترجمه انتساب	۱۲
۱۲	۶.۳ متد visitIfStmt - ترجمه دستور شرطی	۱۲
۱۲	۷.۳ متد visitWhileStmt - ترجمه حلقه while	۱۲
۱۳	۸.۳ متد visitForStmt - ترجمه حلقه for	۱۳
۱۳	۹.۳ متد visitTryStmt - ترجمه ساختار try-catch	۱۳

۱۴	توابع کمکی مهم	۴
۱۴	mapType - نگاشت انواع داده	۱.۴
۱۴	guardDivisionByZero - محافظت از تقسیم بر صفر	۲.۴
۱۴	findZeroDivisors - یافتن مقسوم‌علیه‌های صفر	۳.۴
۱۵	expandPower - تبدیل توان	۴.۴
۱۵	indent - فرمت‌بندی خروجی	۵.۴
۱۵	exitLoopFlags / enterLoopFlags - مدیریت پرچم‌های حلقه	۶.۴

۵ نتیجه‌گیری و جمع‌بندی

۱۷	فاز دوم	۱۷
۱۷	مقدمه فاز دوم	۱.۵
۱۷	برنامه‌های مورد بررسی	۲.۵
۱۷	۱.۲.۵ مدل پایه (output.pml)	
۱۷	۲.۲.۵ برنامه‌های تکمیلی (برای سناریوهای نقض)	
۱۹	۳.۵ خصوصیت اول - Safety	
۱۹	۱.۳.۵ فرمول LTL	
۱۹	۲.۳.۵ چرا فرمول به این شکل است؟	
۲۰	۳.۳.۵ سناریوی تایید - مدل پایه	
۲۰	۴.۳.۵ سناریوی نقض - برنامه (الف)	
۲۱	۴.۵ خصوصیت دوم - Liveness	
۲۱	۱.۴.۵ فرمول LTL	
۲۲	۲.۴.۵ چرا فرمول به این شکل است؟	
۲۲	۳.۴.۵ سناریوی تایید - مدل پایه	
۲۳	۴.۴.۵ سناریوی نقض (تلاش) - برنامه (ج-۱)	
۲۴	۵.۵ خصوصیت سوم - Reachability	
۲۴	۱.۵.۵ فرمول LTL	
۲۵	۲.۵.۵ چرا فرمول به این شکل است؟	
۲۵	۳.۵.۵ سناریوی تایید - مدل پایه (انتظار: نقض یافت شود)	
۲۶	۴.۵.۵ سناریوی مکمل - برنامه (ج-۲) (انتظار: نقض یافت نشود)	
۲۷	۶.۵ خصوصیت چهارم - Deadlock Freedom	
۲۷	۱.۶.۵ دستور اجرا	
۲۷	۲.۶.۵ خروجی ترمینال (مسیر اجرای کامل)	
۲۸	۳.۶.۵ تفسیر	
۲۸	۷.۵ خصوصیت پنجم - Invariant	
۲۸	۱.۷.۵ فرمول LTL	
۲۸	۲.۷.۵ چرا فرمول به این شکل است؟	
۲۸	۳.۷.۵ سناریوی تایید - مدل پایه	
۲۹	۴.۷.۵ سناریوی نقض - برنامه (ب)	

فاز اول

۱.۰ مقدمه

۲.۰ ارتباط با نظریه اتوماتا و زبان‌ها

- یک برنامه در حال اجرا را می‌توان به عنوان یک State Machine در نظر گرفت - هر وضعیت حافظه یک State است و هر دستور، یک Transition.
- خواصی که روی برنامه بررسی می‌شوند، با استفاده از Linear Temporal Logic (LTL) بیان می‌شوند.
- هر فرمول LTL معادل یک Büchi Automaton است - یک ماشین حالت محدود که روی کلمات نامتناهی کار می‌کند.
- ابزار SPIN این اتوماتا را با مدل سیستم ترکیب کرده و تمام حالت‌های ممکن را جستجو می‌کند.
- پس آنچه در این پروژه انجام می‌دهیم، عملاً همان نظریه‌ای است که در کلاس خوانده‌ایم را به صورت عملی و قابل لمس پیاده‌سازی می‌کنیم.

۳.۰ زیرمجموعه زبان Hash پشتیبانی شده

زیرمجموعه مورد نظر شامل موارد زیر است:

- متغیرهای از نوع adad و boole
- دستور شرطی age/bood/vagarna
- حلقه‌های ta و baraye به همراه shekan و edame
- ساختار gereftar/emtehan به صورت پایه (یک بلوک catch بدون تودرتو)
- عملگرهای محاسباتی و مقایسه‌ای تعریف شده در پروژه اول

۴.۰ محدودیت‌های پیاده‌سازی

- دستور shekan تنها از حلقه‌ای که مستقیماً داخل آن است خارج می‌شود، نه از حلقه‌های بیرونی.
- دستور edame اجرای بدنه فعلی را متوقف کرده و به شرط حلقه بازمی‌گردد.

- برای gereftar/emtehan، تنها یک بلوک gereftar پشتیبانی می‌شود و تودرتوی بلوک‌های emtehan در این پروژه لازم نیست.
- کلاس‌ها و متدها جزو این زیرمجموعه نیستند.

۵.۰ نحوه عملکرد کل سیستم

فرآیند اجرای پروژه به صورت زیر است:



در این معماری، ابتدا کد ورودی توسط Lexer به توکن‌ها تبدیل می‌شود. سپس Parser بر اساس گرامر تعریف شده، درخت نحوی (Parse Tree) را تولید می‌کند. پس از اطمینان از عدم وجود خطای نحوی، Semantic Checker قواعد معنایی (مانند تعریف متغیرها قبل از استفاده) را بررسی می‌کند. در نهایت، Visitor درخت را پیمایش کرده و کد معادل Promela تولید می‌کند.

فصل ۱

توضیح کلاس Main

کلاس Main نقطه شروع اجرای برنامه است و وظیفه هماهنگی بین بخش‌های مختلف را بر عهده دارد.

۱.۱ خواندن ورودی

```
CharStream input = CharStreams.fromPath(Paths.get(args[0]));
```

این خط فایل ورودی که به عنوان آرگومان خط فرمان دریافت شده است را به جریان کاراکتری تبدیل می‌کند. این جریان سپس توسط *Lexer* پردازش می‌شود. استفاده از *Paths.get* و *CharStreams.fromPath* روش استاندارد و امن برای خواندن فایل در JAVA است.

۲.۱ تولید *Lexer* و *Parser*

```
HashLexer lexer = new HashLexer(input);  
CommonTokenStream tokens = new CommonTokenStream(lexer);  
HashParser parser = new HashParser(tokens);
```

- *Lexer*: جریان کاراکتری را به توکن‌های معنادار (مانند کلیدواژه‌ها، عملگرها، شناسه‌ها و اعداد) تبدیل می‌کند.
- *TokenStream*: جریان توکن‌ها را به صورتی که *Parser* بتواند از آن استفاده کند، سازماندهی می‌کند.
- *Parser*: بر اساس گرامر تعریف شده در فایل *Hash.g4*، توکن‌ها را تحلیل کرده و درخت نحوی انتزاعی تولید می‌کند.

۳.۱ بررسی خطاهای نحوی

```
if (parser.getNumberOfSyntaxErrors() > 0) {  
    System.out.println("Syntax errors found. Semantic  
        checking skipped.");  
    return;  
}
```

در صورت وجود خطای نحوی، ادامه پردازش متوقف می‌شود. این کار از تولید خروجی ناقص یا اشتباه جلوگیری می‌کند. خطاهای نحوی ممکن است شامل استفاده از کلیدواژه‌های اشتباه، قرارگیری نادرست پرانتزها، یا ساختارهای غیرمجاز باشند.

۴.۱ بررسی معنایی

```
۱ SemanticChecker checker = new SemanticChecker();  
۲ checker.collectDef(tree);  
۳ checker.SemanticCheck(tree);
```

در این مرحله، دو عملیات مهم انجام می‌شود:

۱. **جمع‌آوری تعاریف:** collectDef تمام تعاریف متغیرها، توابع و کلاس‌ها را از درخت استخراج کرده و در یک جدول نماد (Symbol Table) ذخیره می‌کند.

۲. **بررسی معنایی:** SemanticCheck قواعد زیر را بررسی می‌کند:

- استفاده از متغیرها قبل از تعریف
- تطابق نوع داده در عملیات‌ها
- تعریف تکراری متغیرها
- استفاده صحیح از ساختارهای شرطی و حلقه‌ها
- و

اگر خطای معنایی وجود داشته باشد، پیام‌های خطا نمایش داده می‌شوند و برنامه ادامه نمی‌یابد.

۵.۱ تولید کد Promela

```
۱ HashToPromela translator = new HashToPromela();  
۲ String outputCode = translator.visit(tree);
```

در این مرحله، درخت نحوی توسط Visitor پیمایش می‌شود و برای هر گره، کد معادل Promela تولید می‌گردد. استفاده از Visitor به ما امکان می‌دهد تا دقیقاً کنترل کنیم که برای هر نوع گره چه کدی تولید شود.

۶.۱ ذخیره خروجی

```
۱ Path outputDir = Paths.get("src/codes");  
۲ Path outputFile = outputDir.resolve("output.pml");  
۳ Files.writeString(outputFile, outputCode);
```

کد تولید شده در فایل output.pml در پوشه src/codes ذخیره می‌شود. این مسیر استاندارد باعث می‌شود که خروجی به راحتی قابل دسترسی و استفاده در فازهای بعدی باشد.

فصل ۲

چرا از Visitor استفاده شده است؟

۱.۲ مقایسه Visitor و Listener

در این پروژه از الگوی Visitor به جای Listener استفاده شده است. جدول زیر مقایسه جامعی بین این دو الگو ارائه می‌دهد:

ویژگی	Visitor	Listener
مقدار بازگشتی	دارد (می‌تواند هر نوع داده‌ای برگرداند)	ندارد (void)
کنترل پیمایش	کامل (خودتان تصمیم می‌گیرید چه گره‌هایی را ببینید)	محدود (پیمایش خودکار توسط ANTLR)
کاربرد اصلی	تولید کد، تبدیل، تحلیل عمیق	تحلیل ساده، جمع‌آوری اطلاعات
وابستگی به گرامر	کم (هر گره یک متد مجزا دارد)	زیاد (همه در یک متد)
پیچیدگی پیاده‌سازی	بیشتر (نیاز به مدیریت پیمایش)	کمتر (پیمایش خودکار)
قابلیت توسعه	بالا (افزودن عملیات جدید بدون تغییر گره‌ها)	متوسط

جدول ۱.۲: مقایسه الگوهای Visitor و Listener

۲.۲ دلایل اصلی انتخاب Visitor

۱. **تولید مقدار بازگشتی:** مهم‌ترین دلیل، نیاز به تولید کد Promela به عنوان خروجی است. در Visitor، هر متد می‌تواند یک رشته (یا هر نوع داده دیگری) برگرداند، در حالی که Listener خروجی ندارد.
۲. **کنترل کامل بر پیمایش:** در Visitor، خودمان تصمیم می‌گیریم که چه گره‌هایی را بازدید کنیم و به چه ترتیبی. این ویژگی برای ترجمه دقیق دستورات Hash به Promela ضروری است، زیرا ترتیب اجرای دستورات و ساختارهای کنترلی باید به درستی حفظ شود.
۳. **بهبود قابلیت نگهداری:** با استفاده از Visitor، هر قانون گرامر یک متد مجزا دارد که کار اصلاح و توسعه را بسیار ساده‌تر می‌کند. اگر بخواهیم یک دستور جدید اضافه کنیم، فقط یک متد جدید می‌نویسیم.
۴. **مدیریت بهتر Context:** در فرآیند ترجمه، نیاز به نگهداری وضعیت‌هایی مانند برچسب‌های حلقه‌ها یا پرچم‌های استثنا داریم. Visitor با استفاده از متغیرهای عضو کلاس یا پشته‌ها، این کار را به سادگی امکان‌پذیر می‌کند.

۵. انتخابی بازدید از گره‌ها: در Visitor، می‌توانیم از بازدید برخی گره‌ها صرف‌نظر کنیم یا گره‌های خاصی را چندین بار بازدید کنیم، در حالی که در Listener همه گره‌ها دقیقاً یک بار بازدید می‌شوند.

۳.۲ نمونه‌های عملی از مزایای Visitor

- مدیریت تقسیم بر صفر: در متدهای مختلف (مانند visitWhileStmt)، می‌توانیم تصمیم بگیریم که قبل از شرط اصلی حلقه، یک شرط اضافی برای بررسی تقسیم بر صفر اضافه کنیم.
- مدیریت حلقه‌های تو در تو: با استفاده از پشته continueLabels، می‌توانیم به راحتی برچسب‌های حلقه‌های تو در تو را مدیریت کنیم. هر بار که وارد یک حلقه جدید می‌شویم، برچسب آن را به پشته اضافه می‌کنیم و پس از خروج، آن را حذف می‌کنیم.
- مدیریت استثناها: در ساختار try-catch، با استفاده از پرچم‌های استثنا که روی پشته قرار می‌گیرند، می‌توانیم رفتار استثنا را در Promela شبیه‌سازی کنیم.

فصل ۳

طراحی (HashToPromela) Translator

کلاس HashToPromela مسئول اصلی تبدیل کد Hash به Promela است. این کلاس از HashBaseVisitor<String> ارث‌بری کرده و برای هر گره از درخت نحوی، یک رشته خروجی تولید می‌کند.

۱.۳ ساختار کلی کلاس

• متغیرهای سراسری اضافه شده:

- divByZero: برای بررسی ویژگی Safety (جلوگیری از تقسیم بر صفر)
- inLoop و exitLoop: برای بررسی ویژگی‌های Liveness
- endReached: برای بررسی ویژگی Reachability
- activeLoopCount: برای مدیریت حلقه‌های تو در تو

• پشته‌ها:

- continueLabels: ذخیره برچسب‌های حلقه‌ها برای دستور edame
- exceptionFlags: ذخیره پرچم‌های استثنا برای ساختار try-catch
- catchLabels: ذخیره برچسب‌های بلوک‌های catch

• شمارنده‌ها:

- loopCounter: تولید برچسب‌های یکتا برای حلقه‌ها
- exceptionCounter: تولید شناسه‌های یکتا برای استثناها

۲.۳ متد visitProgram - نقطه ورود اصلی

این متد مهم‌ترین متد کلاس است و ساختار کلی فایل خروجی را تولید می‌کند:

۱. تعریف متغیرهای کمکی: در ابتدای خروجی، تمام متغیرهای کمکی که برای بررسی خواص سیستم نیاز هستند، تعریف می‌شوند:

```

1      bool divByZero = false;
2      bool inLoop = false;
3      bool exitLoop = false;
4      bool endReached = false;
5      int activeLoopCount = 0;

```

۲. پردازش متغیرهای سراسری: اعلان‌های سطح بالا که از نوع تعریف متغیر هستند، شناسایی شده و به صورت متغیرهای سراسری در Promela تعریف می‌شوند. این کار باعث می‌شود که متغیرها در کل برنامه قابل دسترسی باشند.

۳. تعریف بدنه اصلی: سایر گره‌های سطح بالا (دستورات، توابع، کلاس‌ها) درون یک پروسه فعال با نام main قرار می‌گیرند، همچنین متغیرهای سراسری در این قسمت مقدار دهی می‌شوند:

```

1      active proctype main() {
2          \\ body
3      }

```

۴. افزودن برچسب پایان: در انتهای بدنه، برچسب endReachedLabel و دستور تنظیم endReached = true اضافه می‌شود تا بتوان ویژگی Reachability را بررسی کرد.

۳.۳ متد visitVarDecl - ترجمه تعریف متغیرها

این متد وظیفه تبدیل اعلان متغیرها را بر عهده دارد:

۱. نگاشت نوع: نوع داده Hash به نوع معادل در Promela تبدیل می‌شود:

- $\text{int} \rightarrow \text{adad}$
- $\text{bool} \rightarrow \text{boole}$
- سایر انواع (مانند matn, harf) پشتیبانی نمی‌شوند

۲. مدیریت عملگرهای افزایش/کاهش: اگر عبارت اولیه شامل ++x یا x++ باشد، به صورت زیر ترجمه می‌شود (به طور مشابه برای --):

- برای ++x: ابتدا مقداردهی با مقدار فعلی، سپس افزایش
- برای x++: ابتدا افزایش، سپس مقداردهی با مقدار جدید

۳. بررسی تقسیم بر صفر: اگر عبارت شامل عملگر تقسیم باشد، تابع guardDivisionByZero فراخوانی می‌شود تا از تقسیم بر صفر جلوگیری کند.

۴.۳ ترجمه عبارات (Expressions) در HashToPromela

۱.۴.۳ روش کلی ترجمه

در کلاس HashToPromela، ترجمه عبارات با استفاده از الگوی Visitor و به صورت بازگشتی انجام می‌شود. به این معنی که هر عبارت بزرگ به زیرعبارت‌های کوچکتر تقسیم شده و هر کدام به صورت مستقل ترجمه می‌شوند، سپس نتایج با هم ترکیب می‌شوند.

۲.۴.۳ سلسله‌مراتب ترجمه عبارات

ترجمه عبارات از بالا به پایین و به ترتیب زیر انجام می‌شود:

visitExpr()	← Entry Point
visitLogicalOrExpr()	← Logical OR ()
visitLogicalAndExpr()	← Logical AND (&&)
visitEqualityExpr()	← Equality (==, !=)
visitRelationalExpr()	← Relational (<, >, <=, >=)
visitAdditiveExpr()	← Addition/Subtraction (+, -)
visitMultiplicativeExpr()	← Multiplication/Division (*, /, %)
visitPowerExpr()	← Power (**)
visitUnaryExpr()	← Unary (!, -, ++, --)
visitPostfixExpr()	← Postfix (++ , --)
visitPrimaryExpr()	← Primary (Numbers, Variables, Parentheses)

۳.۴.۳ نحوه اتصال متدها

هر متد Visitor یک رشته برمی‌گرداند و برای زیرعبارتها، متد visit مناسب را فراخوانی می‌کند. به عنوان مثال، متد visitAdditiveExpr برای ترجمه (a + b) به این صورت عمل می‌کند:

۱. ابتدا visit(ctx.multiplicativeExpr(0)) را صدا می‌زند که به "a" می‌رسد

۲. عملگر "+" را دریافت می‌کند

۳. سپس visit(ctx.multiplicativeExpr(1)) را صدا می‌زند که به "b" می‌رسد

۴. در نهایت "a + b" را برمی‌گرداند

۴.۴.۳ عملگرهای پشتیبانی‌شده

معادل Promela	عملگرهای Hash	دسته
+، -، *، /، (ضرب متوالی)	+، -، *، /، **	حسابی
همان عملگرها	<، <=، >، >=، !=	مقایسه‌ای
!، ، &&	!، ، &	منطقی
x = x - 1، x = x + 1	x --، x++، --x، ++x	یکانی

جدول ۱.۳: عملگرهای پشتیبانی‌شده

۵.۴.۳ مدیریت عملگرهای خاص

• توان (**): با تابع expandPower به ضرب متوالی تبدیل می‌شود:

$$x * x * x \rightarrow x^{**3}$$

• افزایش/کاهش (++، -): ترتیب ارزیابی پیشوند و پسوند حفظ می‌شود:

$$x++ \rightarrow \text{ابتدا مقدار فعلی، سپس افزایش}$$

$$++x \rightarrow \text{ابتدا افزایش، سپس مقدار جدید}$$

• تقسیم بر صفر: با تابع guardDivisionByZero مدیریت می‌شود

۶.۴.۳ مزایای روش Visitor

- کد تمیز و خواناتر: هر عملگر در متد جداگانه
- توسعه‌پذیری بالا: اضافه کردن عملگر جدید با اضافه کردن یک متد
- مدیریت خودکار بازگشت: ANTLR خودش تشخیص می‌دهد کدام متد صدا زده شود

۵.۳ متد visitAssignment - ترجمه انتساب

عملگرهای انتساب ساده و مرکب را پشتیبانی می‌کند:

```
1      switch (op) {
2          case "=": return left + " = " + right;
3          case "+=": return left + " = " + left + " + " + right;
4          case "-=": return left + " = " + left + " - " + right;
5          case "*=": return left + " = " + left + " * " + right;
6          case "/=": return left + " = " + left + " / " + right;
7      }
```

برای عملگر /=، بررسی تقسیم بر صفر اضافه می‌شود.

۶.۳ متد visitIfStmt - ترجمه دستور شرطی

دستور شرطی age/blood/vagarna به ساختار `if :: ... :: else -> ... fi` در Promela تبدیل می‌شود:

```
1      if
2          :: (condition) ->
3          // then block
4          :: else ->
5          // else block (optional)
6      fi
```

همچنین، اگر شرط شامل عملیات تقسیم باشد، یک شرط اضافی برای بررسی تقسیم بر صفر قبل از شرط اصلی اضافه می‌شود.

۷.۳ متد visitWhileStmt - ترجمه حلقه while

حلقه ta به ساختار `do :: ... :: else -> break od` تبدیل می‌شود:

۱. ایجاد برچسب: یک برچسب یکتا برای شروع حلقه تولید می‌شود (مثلاً `loopStartNo_1`).
۲. افزایش شمارنده: `activeLoopCount` یک واحد افزایش می‌یابد.
۳. ترجمه شرط: شرط حلقه به یک شرط در ساختار `do` تبدیل می‌شود.
۴. مدیریت پرچم‌ها: قبل از بدنه حلقه، پرچم‌های `inLoop` و `exitLoop` تنظیم می‌شوند.
۵. بررسی تقسیم بر صفر: یک شرط اضافی برای تشخیص تقسیم بر صفر قبل از شرط اصلی اضافه می‌شود.
۶. پس از خروج: پس از خروج از حلقه، `activeLoopCount` کاهش یافته و پرچم‌ها به‌روز می‌شوند.

۸.۳ متد visitForStmt - ترجمه حلقه for

حلقه baraye به ساختار زیر در Promela تبدیل می‌شود:

```
۱ // Initialization
۲ activeLoopCount = activeLoopCount + 1;
۳ do
۴ :: (condition) ->
۵ // Set flags
۶ // Loop body
۷ updateLabel:
۸ // Update
۹ :: else -> break
۱۰ od
۱۱ // Reset flags after exit
```

۹.۳ متد visitTryStmt - ترجمه ساختار try-catch

از آنجا که Promela از مفهوم استثنا پشتیبانی نمی‌کند، این ساختار با استفاده از پرچم‌ها و برچسب‌ها شبیه‌سازی می‌شود:

۱. تعریف پرچم: یک پرچم جدید مانند errFlag_1 با مقدار false تعریف می‌شود.
۲. اجرای بلوک try: بدنه emtehan اجرا می‌شود. در صورت بروز خطا (تقسیم بر صفر)، پرچم به true تغییر کرده و به برچسب catchCheck_1 پرش می‌شود.
۳. بررسی پرچم: پس از بلوک try، پرچم بررسی می‌شود و اگر true باشد، بلوک catch اجرا می‌شود.

فصل ۴

توابع کمکی مهم

۱.۴ mapType - نگاشت انواع داده

تبدیل انواع داده Hash به Promela:

```
۱ private String mapType(String hashType) {
۲     switch (hashType) {
۳         case "adad": return "int";
۴         case "boole": return "bool";
۵         default:     return "";
۶     }
۷ }
```

۲.۴ guardDivisionByZero - محافظت از تقسیم بر صفر

این تابع با دریافت لیست مقسوم‌علیه‌های مشکوک، یک ساختار شرطی تولید می‌کند:

```
۱ if
۲ :: (divisor1 == 0 || divisor2 == 0) ->
۳ divByZero = true;
۴ //      try-catch
۵ errFlag = true;
۶ goto catchLabel;
۷ :: else ->
۸ //
۹ fi
```

۳.۴ findZeroDivisors - یافتن مقسوم‌علیه‌های صفر

این تابع با پیمایش بازگشتی درخت عبارت، تمام مقسوم‌علیه‌های عملیات تقسیم یا باقی‌مانده را جمع‌آوری می‌کند. به این صورت که در گره‌های MultiplicativeExprContext (شامل ضرب، تقسیم و باقی‌مانده)، عملگرها را بررسی کرده و اگر عملگر / یا % باشد، عبارت سمت راست را به عنوان یک مقسوم‌علیه مشکوک به لیست اضافه می‌کند. برای سایر گره‌ها، به صورت بازگشتی به فرزندان رفته و جستجو را ادامه می‌دهد. برای مثال، عبارت $(a / b / c) + (e / d)$ خروجی ["b", "c", "d"] را تولید

می‌کند (چون در $a/b/c$ ، مقسوم‌علیه‌های b و c هر دو جمع‌آوری می‌شوند). این لیست سپس برای ساخت شرط بررسی صفر بودن مقسوم‌علیه‌ها استفاده می‌شود. این تابع در متدهای `visitVarDecl`، `visitIfStmt`، `visitAssignmentStmt` و `visitWhileStmt` برای محافظت از تقسیم بر صفر به کار می‌رود.

۴.۴ expandPower - تبدیل توان

از آنجا که Promela از عملگر توان پشتیبانی نمی‌کند، $x^{**}n$ به $x*x*x*...$ (تکرار n بار) تبدیل می‌شود:

```
1 private String expandPower(String base, int exponent) {
2     StringBuilder sb = new StringBuilder("");
3     for (int i = 0; i < exponent; i++) {
4         if (i > 0) sb.append(" * ");
5         sb.append("(").append(base).append(")");
6     }
7     sb.append(")");
8     return sb.toString();
9 }
```

۵.۴ indent - فرمت‌بندی خروجی

برای افزایش خوانایی کد خروجی، به هر خط ۴ فاصله اضافه می‌کند:

```
1 private String indent(String code) {
2     StringBuilder out = new StringBuilder();
3     for (String line : code.split("\n")) {
4         if (!line.isBlank()) {
5             out.append("    ").append(line);
6         }
7         out.append("\n");
8     }
9     return out.toString();
10 }
```

۶.۴ exitLoopFlags / enterLoopFlags - مدیریت پرچم‌های حلقه

این توابع کد مربوط به تنظیم و بازنشانی پرچم‌های `inLoop` و `exitLoop` را تولید می‌کنند که برای بررسی ویژگی‌های Liveness ضروری هستند.

فصل ۵

نتیجه‌گیری و جمع‌بندی

در این پروژه، یک ترنسلیتور کامل و کارآمد از زبان آموزشی Hash به زبان Promela طراحی و پیاده‌سازی شد. دستاوردهای اصلی این فاز عبارتند از:

۱. **ترجمه دقیق:** تمام ساختارهای زبانی مورد نیاز به درستی به Promela ترجمه می‌شوند.
۲. **مدیریت خواص سیستم:** با افزودن متغیرهای کمکی مانند `divByZero`، `inLoop` و `endReached`، زیرساخت لازم برای بررسی خواص مختلف در فازهای بعدی فراهم شده است.
۳. **پشتیبانی از استثناها:** با وجود اینکه Promela از استثنا پشتیبانی نمی‌کند، با استفاده از تکنیک‌های شبیه‌سازی، ساختار `try-catch` پشتیبانی می‌شود.
۴. **کد خوانا و قابل نگهداری:** استفاده از الگوی `Visitor` و توابع کمکی مناسب، کد را خوانا، ماژولار و به راحتی قابل توسعه کرده است.
۵. **پشتیبانی از عملگرهای پیشرفته:** عملگرهای توان، افزایش/کاهش، و انتساب مرکب به درستی ترجمه می‌شوند.

خروجی تولید شده در این فاز (فایل `output.pml`) به عنوان ورودی فاز دوم (Model Checking) استفاده خواهد شد. در فاز دوم، با استفاده از ابزار SPIN، پنج ویژگی مختلف روی این مدل بررسی می‌شوند تا صحت سیستم تأیید یا رد شود.

• **ویژگی Safety:** جلوگیری از تقسیم بر صفر (`divByZero`)

• **ویژگی Liveness:** خروج از حلقه‌ها (`exitLoop → inLoop`)

• **ویژگی Reachability:** رسیدن به پایان برنامه (`endReached`)

• **ویژگی Freedom Deadlock:** عدم وقوع بن‌بست

• **ویژگی Invariant:** همواره غیرمنفی بودن یک متغیر

این پروژه نشان می‌دهد که چگونه مفاهیم تئوری زبان‌ها و ماشین‌ها (اتوماتا، گرامرها، و معناشناسی) در عمل و برای تأیید صحت نرم‌افزارهای واقعی به کار گرفته می‌شوند. توانایی اثبات ریاضی درستی نرم‌افزارها، یکی از مهارت‌های کلیدی در صنعت نرم‌افزار مدرن است.

فاز دوم

۱.۵ مقدمه فاز دوم

در فاز اول، برنامه‌ی زبان Hash به مدل Promela ترجمه شد (فایل output.pml). هدف این فاز، بررسی پنج خصوصیت مشخص – Invariant و Deadlock Freedom، Reachability، Liveness، Safety – بر روی این مدل با استفاده از ابزار SPIN است. برای هر خصوصیت، فرمول LTL متناظر (در صورت نیاز)، دستور دقیق اجرا، خروجی ترمینال و یک تفسیر مستقل ارائه شده است. تمامی خصوصیات با فرمول‌های named در فایل properties.ltl تعریف شده‌اند و با دستور `spin -search -ltl <name> verify.pml` اجرا شده‌اند.

روش‌شناسی بررسی. یکی از نکات مهمی که در طول این فاز به آن رسیدیم این است که تأیید یک خصوصیت روی یک برنامه‌ی ثابت، به تنهایی کافی نیست. اگر مسیر خطا یا مسیر بحرانی در آن برنامه‌ی خاص از اساس غیرقابل دسترس باشد، نتیجه‌ی 0 errors که SPIN برمی‌گرداند بدیهی (vacuously true) خواهد بود و چیزی درباره‌ی درستی واقعی مکانیزم تحت بررسی ثابت نمی‌کند. برای پرهیز از این مشکل، برای هر یک از چهار خصوصیت Safety، Liveness، Reachability و Invariant، دو سناریو طراحی و اجرا شده است:

- **سناریوی تایید:** روی مدل پایه (برنامه‌ی اصلی فاز اول)، که در آن انتظار می‌رود خصوصیت برقرار باشد.

- **سناریوی نقض:** روی یک برنامه‌ی طراحی شده که مسیر بحرانی در آن واقعاً reachable است، تا نشان داده شود SPIN می‌تواند نقض واقعی را هم تشخیص دهد (نه اینکه صرفاً همیشه pass بدهد).

خصوصیت Deadlock Freedom از این قاعده مستثناست؛ دلیل آن در بخش مربوطه توضیح داده شده است.

۲.۵ برنامه‌های مورد بررسی

۱.۲.۵ مدل پایه (output.pml)

مدل Promela اصلی که از ترجمه‌ی برنامه‌ی فاز اول به دست آمده، شامل یک عملیات تقسیم محافظت شده با سازوکار gereftar/emtehan (با $y=2$ ثابت)، دو حلقه‌ی متوالی و کراندار (اولی با شرط $i < 5$ ، دومی با شرط $j < 3$)، و یک برچسب پایانی endReachedLabel است. متغیرهای کنترلی inLoop، divByZero، exitLoop و endReached برای امکان بیان پنج خصوصیت در سطح سراسری (global) تعریف شده‌اند.

۲.۲.۵ برنامه‌های تکمیلی (برای سناریوهای نقض)

برای اینکه هر خصوصیت به صورت غیر بدیهی هم بررسی شود، سه برنامه‌ی Hash دیگر طراحی و به طور جداگانه به Promela ترجمه و با SPIN بررسی شدند:

برنامه (الف) – تقسیم بر صفر مستقیم (برای نقض Safety):

```

1      adad x = 10;
2      adad y = 0;
3      adad safe = 0;
4
5      emtehan {
6          safe = x / y;
7      }
8      gereftar (SefrBood e) {
9          safe = 0;
10     }
11
12     bechap(safe);
13

```

برنامه (ب) – کاهش پیوسته‌ی متغیر (برای نقض Invariant):

```

1      adad x = 3;
2      adad i = 0;
3
4      ta (i < 10) {
5          x = x - 1;
6          i = i + 1;
7      }
8
9      bechap(x);
10

```

برنامه (ج-۱) – حلقه‌ی نامتناهی با متغیر متغیر (برای نقض Liveness):

```

1      adad x = 10;
2      adad y = 2;
3      adad i = 0;
4      adad safe = 0;
5
6      ta (i < 5) {
7          x = x - 1;
8      }
9
10     baraye (adad j = 0; j < 3; j = j) {
11         safe = safe + j;
12     }
13
14     bechap(safe);
15

```

در این برنامه، شرط حلقه‌ی اول ($i < 5$) هیچ‌گاه به false نمی‌رسد چون i هرگز افزایش نمی‌یابد؛ بنابراین اجرای برنامه برای همیشه در حلقه‌ی اول باقی می‌ماند و به بدنه‌ی حلقه‌ی دوم یا خط bechap هرگز نمی‌رسد. نکته‌ی مهم اینجا است که در هر تکرار، x یک واحد کم می‌شود؛ یعنی علاوه بر نامتناهی بودن حلقه، مقدار متغیر x هم هیچ‌گاه تکرار نمی‌شود (به سمت منفی بی‌نهایت می‌رود).

برنامه (ج-۲) – حلقه‌ی نامتناهی با state ثابت (برای بررسی مکمل Reachability):

```

1      adad i = 0;
2      adad safe = 0;

```

```

3      adad x = 0;
4
5      ta (i < 5) {
6          safe = safe;
7      }
8
9      baraye (adad j = 0; j < 3; j = j) {
10         safe = safe + j;
11     }
12
13     bechap(safe);
14

```

این برنامه هم مثل (ج-۱) در حلقه‌ی اول برای همیشه باقی می‌ماند، با این تفاوت مهم که بدنه‌ی حلقه (safe = safe;) هیچ متغیری را واقعاً تغییر نمی‌دهد؛ در نتیجه state کلی برنامه پس از اولین تکرار دقیقاً تکرار می‌شود (self-loop).

۳.۵ خصوصیت اول – Safety

۱.۳.۵ فرمول LTL

$$\Box \neg(\text{divByZero})$$

معادل آن در properties.ltl: `ltl noDivByZero: [] !(divByZero)`

۲.۳.۵ چرا فرمول به این شکل است؟

خصوصیت‌های Safety معمولاً به صورت $\Box \neg(\text{bad})$ بیان می‌شوند، زیرا هدف آن‌ها تضمین این است که سیستم در هیچ‌یک از حالت‌های قابل دسترس خود وارد وضعیت نامطلوب (bad state) نشود. در این پروژه، وقوع تقسیم بر صفر به‌عنوان وضعیت نامطلوب در نظر گرفته شده است؛ بنابراین از فرمول

$$\Box(\neg \text{divByZero})$$

استفاده شده است.

اگر به‌جای عملگر $\Box$ از $\Diamond$ استفاده می‌شد، یعنی

$$\Diamond(\neg \text{divByZero})$$

این فرمول تنها بیان می‌کرد که حداقل یک حالت در اجرای سیستم وجود دارد که در آن تقسیم بر صفر رخ نداده است. بدیهی است چنین شرطی حتی در صورتی که سیستم در ادامه اجرا وارد حالت خطا شود نیز برقرار خواهد بود؛ زیرا وجود تنها یک حالت بدون خطا برای صدق این فرمول کافی است. در مقابل، عملگر $\Box$ بیان می‌کند که شرط مورد نظر باید در تمام حالت‌های قابل دسترس برقرار باشد. از این‌رو، این عملگر انتخاب مناسبی برای بیان ویژگی‌های ایمنی است و تضمین می‌کند که در هیچ مرحله‌ای از اجرای سیستم تقسیم بر صفر رخ نخواهد داد.

۳.۳.۵ سناریوی تایید – مدل پایه

دستور اجرا

```
spin -search -ltl noDivByZero verify.pml
```

خروجی ترمینال

```
1      (Spin Version 6.5.1 -- 31 July 2020)
2      + Partial Order Reduction
3
4      Full statespace search for:
5      never claim          + (noDivByZero)
6      assertion violations + (if within scope of claim)
7      acceptance cycles   + (fairness disabled)
8      invalid end states  - (disabled by never claim)
9
10     State-vector 48 byte, depth reached 137, errors: 0
11     69 states, stored
12     1 states, matched
13     70 transitions (= stored+matched)
14     0 atomic steps
15     hash conflicts: 0 (resolved)
16
17     unreachable in proctype main
18     output.pml:20, state 7, "divByZero = 1"
19     output.pml:21, state 8, "errFlag_1 = 1"
20     output.pml:29, state 15, "safe = 0"
21     (6 of 69 states)
22     unreachable in claim noDivByZero
23     _spin_nvr.tmp:8, state 10, "-end-"
24     (1 of 10 states)
25
26     pan: elapsed time 0.003 seconds
27
```

تفسیر

جستجوی کامل فضای حالت هیچ نقضی برای فرمول $\neg(\text{divByZero})$ پیدا نکرد (errors: 0). با این حال، بخش unreachable نشان می‌دهد خط $\text{divByZero} = 1$ اصلاً کاوش نشده است؛ چون در این مدل $y=2$ ثابت است و هیچ مسیری برای رسیدن آن به صفر وجود ندارد. بنابراین این نتیجه به‌تنهایی بدیهی است و برای اطمینان از درستی واقعی گارد تقسیم، سناریوی نقض زیر لازم است.

۴.۳.۵ سناریوی نقض – برنامه (الف)

دستور اجرا

```
spin -search -ltl noDivByZero verify.pml
```

خروجی ترمینال

```
1 pan: ltl formula noDivByZero
2 pan:1: assertion violated !( !(divByZero))) (at depth 12)
3 pan: wrote verify.pml.trail
4
5 (Spin Version 6.5.1 -- 31 July 2020)
6 Warning: Search not completed
7 + Partial Order Reduction
8
9 Full statespace search for:
10 never claim + (noDivByZero)
11 assertion violations + (if within scope of claim)
12 acceptance cycles + (fairness disabled)
13 invalid end states - (disabled by never claim)
14
15 State-vector 32 byte, depth reached 12, errors: 1
16 7 states, stored
17 0 states, matched
18 7 transitions (= stored+matched)
19 0 atomic steps
20 hash conflicts: 0 (resolved)
21
22 pan: elapsed time 0.002 seconds
23
```

تفسیر

در این برنامه $y=0$ از ابتدا مقداردهی شده، پس شاخه‌ی گارد بلافاصله فعال می‌شود. SPIN در عمق ۱۲ (تنها ۷ حالت) یک نقض واقعی برای `noDivByZero` پیدا کرد (errors: 1)، یعنی `divByZero` به `true` تغییر کرده است. این نتیجه دقیقاً همان چیزی است که انتظار می‌رفت: وقتی مسیر خطرناک واقعاً `reachable` است، SPIN آن را تشخیص می‌دهد. مجموع دو سناریو نشان می‌دهد که فلگ `divByZero` در مدل پایه به این دلیل هرگز `true` نشد که مسیر خطا در آن برنامه‌ی خاص وجود نداشت، نه به این دلیل که مکانیزم گارد ناقص است.

۴.۵ خصوصیت دوم – Liveness

۱.۴.۵ فرمول LTL

$$\Box (inLoop \rightarrow \Diamond exitLoop)$$

معادل آن در `properties.ltl`: `ltl loopExit: [] ((! (inLoop)) || (<> (exitLoop)))`

۲.۴.۵ چرا فرمول به این شکل است؟

خصوصیت‌های Liveness معمولاً به صورت $\Box(P \rightarrow \Diamond Q)$ بیان می‌شوند. مفهوم این فرمول آن است که هرگاه شرط P برقرار شود، سیستم موظف است در ادامه اجرای خود نهایتاً به حالتی برسد که Q برقرار باشد.

در پروژه حاضر، پس از ورود به یک حلقه، انتظار می‌رود که اجرای برنامه در نهایت از آن حلقه خارج شود. به همین دلیل از فرمول

$$\Box(inLoop \rightarrow \Diamond exitLoop)$$

استفاده شده است.

اگر تنها از عملگر $\Diamond$ استفاده می‌شد، مانند

$$\Diamond(exitLoop)$$

این فرمول فقط بیان می‌کرد که «در یکی از مسیرهای اجرا ممکن است از حلقه خارج شویم» و هیچ تضمینی درباره تمام دفعات ورود به حلقه ارائه نمی‌داد. استفاده همزمان از $\Box$ و $\Diamond$ تضمین می‌کند که خروج از حلقه برای تمام دفعات ورود به آن در نهایت رخ خواهد داد.

۳.۴.۵ سناریوی تایید – مدل پایه

دستور اجرا

```
spin -search -ltl loopExit verify.pml
```

خروجی ترمینال

```
1 (Spin Version 6.5.1 -- 31 July 2020)
2 + Partial Order Reduction
3
4 Full statespace search for:
5 never claim          + (loopExit)
6 assertion violations  + (if within scope of claim)
7 acceptance cycles    + (fairness disabled)
8 invalid end states    - (disabled by never claim)
9
10 State-vector 48 byte, depth reached 137, errors: 0
11 110 states, stored (151 visited)
12 77 states, matched
13 228 transitions (= visited+matched)
14 0 atomic steps
15 hash conflicts: 0 (resolved)
16
17 unreachable in proctype main
18 output.pml:20, state 7, "divByZero = 1"
19 output.pml:21, state 8, "errFlag_1 = 1"
20 output.pml:29, state 15, "safe = 0"
21 (6 of 69 states)
```

```

22      unreachable in claim loopExit
23      _spin_nvr.tmp:19, state 13, "-end-"
24      (1 of 13 states)
25
26      pan: elapsed time 0.003 seconds
27

```

تفسیر

هیچ اجرایی پیدا نشد که در آن inLoop برقرار شود ولی exitLoop هرگز به دنبالش نیاید (errors: 0). چون هر دو حلقه‌ی مدل پایه ($i < 5$ و $j < 3$) کراندارند و همیشه با break خاتمه می‌یابند، این نتیجه تا حدی بدیهی است: در این برنامه‌ی خاص اساساً امکان ورود به یک حلقه‌ی نامتناهی وجود ندارد. برای بررسی غیر بدیهی، سناریوی نقض زیر با برنامه (ج) طراحی شد.

۴.۴.۵ سناریوی نقض (تلاش) – برنامه (ج-۱)

دستور اجرا

```
spin -search -ltl loopExit example/liveness.pml
```

خروجی ترمینال

```

1      pan: ltl formula loopExit
2      error: max search depth too small
3
4      (Spin Version 6.5.1 -- 31 July 2020)
5      + Partial Order Reduction
6
7      Full statespace search for:
8      never claim          + (loopExit)
9      assertion violations  + (if within scope of claim)
10     acceptance cycles    + (fairness disabled)
11     invalid end states    - (disabled by never claim)
12
13     State-vector 44 byte, depth reached 9999, errors: 0
14     9992 states, stored (14984 visited)
15     9986 states, matched
16     24970 transitions (= visited+matched)
17     0 atomic steps
18     hash conflicts: 1 (resolved)
19
20     unreachable in proctype main
21     example/liveness.pml:26, state 15, "activeLoopCount = (
activeLoopCount-1)"
22     example/liveness.pml:36, state 25, "activeLoopCount = (
activeLoopCount+1)"
23     example/liveness.pml:41, state 29, "safe = (safe+j)"

```



```

24 example/liveness.pml:54, state 44, "printf('%d\n',safe)"
25 example/liveness.pml:56, state 45, "endReached = 1"
26 example/liveness.pml:58, state 47, "-end-"
27 (19 of 47 states)
28 unreachable in claim loopExit
29 _spin_nvr.tmp:19, state 13, "-end-"
30 (1 of 13 states)
31
32 pan: elapsed time 0.011 seconds
33 pan: rate 1362181.8 states/second
34

```

تفسیر

این اجرا با پیغام error: max search depth too small یعنی عمق جستجوی پیش فرض SPIN (پارامتر -m، با مقدار پیش فرض ۹۹۹۹) قبل از تکمیل جستجوی تودرتو (nested DFS) برای یافتن accepting cycle تمام شد. بنابراین errors: 0 در اینجا به هیچ وجه به معنای تأیید خصوصیت نیست؛ نتیجه صرفاً ناقص است.

با این حال، بخش unreachable شاهد غیرمستقیم قوی‌ای ارائه می‌دهد: تمام دستورات بعد از حلقه‌ی اول (شامل کل حلقه‌ی دوم، printf و برچسب پایانی) در طول ۹۹۹۲ حالت کاوش شده هرگز اجرا نشده‌اند. این کاملاً با طراحی برنامه سازگار است: چون i هرگز افزایش نمی‌یابد، اجرا برای همیشه در حلقه‌ی اول گیر می‌کند و inLoop بدون رسیدن متناظر exitLoop برقرار می‌ماند. دلیل فنی رشد بزرگ فضای حالت (۹۹۹۲ حالت به جای یک self-loop کوچک) هم قابل توضیح است: در بدنه‌ی حلقه در این برنامه، دستور $x = x - 1$ در هر تکرار اجرا می‌شود، پس علاوه بر نامتناهی بودن خود حلقه، مقدار x هم هرگز تکرار نمی‌شود (هر تکرار یک state برداری جدید تولید می‌کند، چون x جزئی از بردار حالت است). در نتیجه SPIN مجبور است پیش از رسیدن به یک حالت تکراری، مسیر رو به رشدی از حالت‌های یکتا را کاوش کند که در عمل به سقف عمق پیش فرض (-m) 9999 برخورد می‌کند. برای دریافت یک counterexample رسمی، لازم است جستجو با عمق بزرگ‌تر تکرار شود؛ این مورد به عنوان محدودیت شناخته شده و کار آتی ثبت می‌شود.

۵.۵ خصوصیت سوم – Reachability

۱.۵.۵ فرمول LTL

فرمولی که به SPIN داده می‌شود، نقیض فرمول اصلی است:

$$\neg(\Diamond \text{endReached})$$

معادل آن در properties.ltl: `ltl reachEnd: ! (<> (endReached))`
نکته‌ی تفسیری مهم: چون فرمول به صورت نقیض داده می‌شود، برخلاف بقیه‌ی خصوصیات، در این مورد یافتن یک نقض (errors: 1) به معنای تأیید reachability است (یعنی مسیری به endReached واقعاً وجود دارد)، و عدم یافتن نقض (errors: 0) به معنای آن است که endReached در آن مدل خاص غیرقابل دسترس است.

۲.۵.۵ چرا فرمول به این شکل است؟

در SPIN معمولاً خصوصیت‌های دسترس‌پذیری به صورت مستقیم بیان نمی‌شوند، زیرا LTL ذاتاً برای بیان خواصی که باید در تمام مسیرهای اجرا برقرار باشند طراحی شده است. برای بررسی اینکه آیا حالت پایانی برنامه قابل دسترسی است، نقیض این ویژگی نوشته می‌شود:

$$\neg \Diamond(endReached)$$

در صورت نقض این فرمول توسط SPIN، یک مسیر نمونه ارائه می‌شود که در آن متغیر `endReached` فعال شده است. بنابراین شکست این فرمول به معنای آن است که حالت پایانی برنامه واقعاً قابل دستیابی بوده و ویژگی `Reachability` برقرار است. به همین دلیل در این پروژه از نقیض ویژگی دسترس‌پذیری استفاده شده است.

۳.۵.۵ سناریوی تایید – مدل پایه (انتظار: نقض یافت شود)

دستور اجرا

```
spin -search -ltl reachEnd verify.pml
```

خروجی ترمینال

```
1 pan: ltl formula reachEnd
2 pan:1: assertion violated !(endReached) (at depth 132)
3 pan: wrote verify.pml.trail
4
5 (Spin Version 6.5.1 -- 31 July 2020)
6 Warning: Search not completed
7 + Partial Order Reduction
8
9 Full statespace search for:
10 never claim + (reachEnd)
11 assertion violations + (if within scope of claim)
12 acceptance cycles + (fairness disabled)
13 invalid end states - (disabled by never claim)
14
15 State-vector 48 byte, depth reached 132, errors: 1
16 67 states, stored
17 0 states, matched
18 67 transitions (= stored+matched)
19 0 atomic steps
20 hash conflicts: 0 (resolved)
21
22 pan: elapsed time 0.003 seconds
23
```

تفسیر

SPIN در عمق ۱۳۲ یک نقض برای فرمول نقیض شده پیدا کرد (errors: 1)؛ یعنی مسیر اجرایی یافت که در آن endReached برقرار می‌شود. طبق نکته‌ی بالا، این نتیجه دقیقاً همان چیزی است که انتظار می‌رفت: مدل پایه یک برنامه‌ی خطی و deterministic است که همیشه به انتهای خود می‌رسد، پس برچسب پایانی حتماً باید reachable باشد. یافتن این counterexample تأیید می‌کند که خصوصیت Reachability برقرار است.

۴.۵.۵ سناریوی مکمل – برنامه (ج-۲) (انتظار: نقض یافت نشود)

برای اینکه نشان دهیم SPIN می‌تواند بین «قابل دسترس» و «غیرقابل دسترس» تمایز واقعی قائل شود (نه اینکه صرفاً همیشه یک counterexample پیدا کند)، همین بررسی روی برنامه (ج-۲) که حلقه‌ی اولش نامتناهی است تکرار شد. توجه شود که این برنامه با برنامه (ج-۱) که برای Liveness استفاده شد متفاوت است؛ در (ج-۲) بدنه‌ی حلقه هیچ متغیری را تغییر نمی‌دهد (safe = safe)، به همین دلیل state برنامه خیلی زود تکرار می‌شود و جستجو، برخلاف (ج-۱)، به‌طور کامل و بدون برخورد به سقف عمق به پایان می‌رسد.

دستور اجرا

```
spin -search -ltl reachEnd example/reachability.pml
```

خروجی ترمینال

```
1 pan: ltl formula reachEnd
2
3 (Spin Version 6.5.1 -- 31 July 2020)
4 + Partial Order Reduction
5
6 Full statespace search for:
7 never claim + (reachEnd)
8 assertion violations + (if within scope of claim)
9 acceptance cycles + (fairness disabled)
10 invalid end states - (disabled by never claim)
11
12 State-vector 44 byte, depth reached 19, errors: 0
13 10 states, stored
14 1 states, matched
15 11 transitions (= stored+matched)
16 0 atomic steps
17 hash conflicts: 0 (resolved)
18
19 unreached in proctype main
20 example/reachability.pml:24, state 14, "activeLoopCount = (
activeLoopCount-1)"
21 example/reachability.pml:39, state 28, "safe = (safe+j)"
22 example/reachability.pml:52, state 43, "printf('%d\n',safe)"
23 example/reachability.pml:54, state 44, "endReached = 1"
```

```

24 example/reachability.pml:56, state 46, "-end-"
25 (19 of 46 states)
26 unreachable in claim reachEnd
27 _spin_nvr.tmp:28, state 10, "-end-"
28 (1 of 10 states)
29
30 pan: elapsed time 0.006 seconds
31

```

تفسیر

این بار جستجو به طور کامل تمام شد (بدون خطای عمق، چون خود فرمول reachEnd با فضای حالت کوچکتری قابل رد کردن است) و هیچ نقضی یافت نشد (errors: 0). بخش unreachable به وضوح نشان می‌دهد خط $\text{endReached} = 1$ هرگز اجرا نشده است. این با تحلیل دستی برنامه کاملاً سازگار است: چون حلقه‌ی اول هرگز پایان نمی‌یابد، اجرا هرگز به بخش دوم برنامه یا برچسب پایانی نمی‌رسد. مقایسه‌ی این نتیجه با سناریوی قبلی (مدل پایه) نشان می‌دهد SPIN واقعاً بین قابل دسترس بودن و نبودن endReached فرق می‌گذارد، نه اینکه نتیجه‌اش وابسته به یک الگوی ثابت باشد.

۶.۵ خصوصیت چهارم – Deadlock Freedom

این خصوصیت نیازی به فرمول LTL ندارد و با قابلیت توکار SPIN بررسی می‌شود. برخلاف چهار خصوصیت دیگر، اینجا سناریوی نقض طراحی نشده است؛ دلیل آن در پایان این بخش توضیح داده شده.

۱.۶.۵ دستور اجرا

```

1 spin -search -deadlock verify.pml
2

```

۲.۶.۵ خروجی ترمینال (مسیر اجرای کامل)

از آنجا که مدل پایه کاملاً deterministic است، SPIN تنها یک مسیر اجرایی یکتا برای proctype main می‌یابد:

```

1 proctype main
2   state 1 -> state 2   verify.pml:13  x = 10
3   state 2 -> state 3   verify.pml:14  y = 2
4   state 3 -> state 4   verify.pml:15  i = 0
5   state 4 -> state 5   verify.pml:16  safe = 0
6   state 5 -> state 12  verify.pml:18  errFlag_1 = 0
7   state 12 -> state 11 verify.pml:19  else (y != 0)
8   state 11 -> state 18 verify.pml:24  safe = (x / y)
9   state 18 -> state 17 verify.pml:28  else (!errFlag_1)
10  state 20 -> state 28 verify.pml:32  activeLoopCount = activeLoopCount + 1
11  state 28 -> state 22 verify.pml:35  (i < 5)
12  state 28 -> state 31 verify.pml:35  else (i >= 5)
13  state 39 -> state 41 verify.pml:51  j = 0
14  state 41 -> state 49 verify.pml:52  activeLoopCount = activeLoopCount + 1
15  state 49 -> state 43 verify.pml:54  (j < 3)
16  state 49 -> state 52 verify.pml:54  else (j >= 3)
17  state 64 -> state 61 verify.pml:71  (x >= 0)

```

```

18 state 61 -> state 66 verify.pml:72 safe = safe + 1
19 state 66 -> state 67 verify.pml:76 printf('%d\n', safe)
20 state 67 -> state 68 verify.pml:78 endReached = 1
21 state 68 -> state 69 verify.pml:79 (1)
22 state 69 -> state 0 verify.pml:80 -end-
23

```

۳.۶.۵ تفسیر

مسیر اجرا بدون هیچ توقف غیرمنتظره‌ای به برچسب -end- می‌رسد که یک پایان معتبر (valid end) state است؛ بنابراین مدل عاری از deadlock است.

چرا سناریوی نقض طراحی نشد؟ طبق قوانین ترجمه‌ی پیاده‌سازی‌شده در ترنسلیتور، هر if/fi تولیدشده (چه از age/vagarna، چه از گاردهای تقسیم، چه از چک errFlag) به‌صورت خودکار یک شاخه‌ی skip -> else دارد، و هر do/od هم شاخه‌ی break -> else صریح دارد. بنابراین به‌صورت ساختاری هیچ حالتی در خروجی این ترنسلیتور وجود ندارد که در آن هیچ transition ای فعال نباشد؛ عاری‌بودن از deadlock نتیجه‌ی مستقیم این تصمیم طراحی است، نه یک ویژگی تصادفی که فقط با شانس در یک تست خاص برقرار شده باشد. تولید یک نقض واقعی مستلزم دستکاری دستی خروجی Promela (مثلاً حذف عمدی یک شاخه‌ی else) است که دیگر محصول ترنسلیتور نیست، بلکه یک Promela دستی و مصنوعی خواهد بود؛ به همین دلیل چنین سناریویی در این گزارش گنجانده نشد.

۷.۵ خصوصیت پنجم – Invariant

۱.۷.۵ فرمول LTL

$$\Box (x \geq 0)$$

معادل آن در properties.ltl: `ltl nonNegativeX: [] ((x>=0))`

۲.۷.۵ چرا فرمول به این شکل است؟

Invariant یا خصوصیتی است که باید در تمام طول اجرای سیستم برقرار بماند. چنین ویژگی‌هایی نیز همانند خصوصیت‌های ایمنی با عملگر $\Box$ بیان می‌شوند. در این پروژه فرض شده است که مقدار متغیر x نباید هیچ‌گاه منفی شود؛ بنابراین فرمول زیر استفاده شده است:

$$\Box (x \geq 0)$$

اگر تنها از عملگر $\Diamond$ استفاده می‌شد، فرمول صرفاً بیان می‌کرد که در یکی از حالت‌های اجرا مقدار متغیر غیرمنفی است، در حالی که هدف invariant تضمین برقرار بودن این شرط در تمام حالات قابل دسترس است.

۳.۷.۵ سناریوی تایید – مدل پایه

دستور اجرا

```
spin -search -ltl nonNegativeX verify.pml
```

```

1      (Spin Version 6.5.1 -- 31 July 2020)
2      + Partial Order Reduction
3
4      Full statespace search for:
5      never claim          + (nonNegativeX)
6      assertion violations  + (if within scope of claim)
7      acceptance cycles    + (fairness disabled)
8      invalid end states   - (disabled by never claim)
9
10     State-vector 48 byte, depth reached 137, errors: 0
11     69 states, stored
12     1 states, matched
13     70 transitions (= stored+matched)
14     0 atomic steps
15     hash conflicts: 0 (resolved)
16
17     unreached in proctype main
18     output.pml:20, state 7, "divByZero = 1"
19     output.pml:21, state 8, "errFlag_1 = 1"
20     output.pml:29, state 15, "safe = 0"
21     (6 of 69 states)
22     unreached in claim nonNegativeX
23     _spin_nvr.tmp:37, state 10, "-end-"
24     (1 of 10 states)
25
26     pan: elapsed time 0.005 seconds
27

```

تفسیر

جستجوی کامل هیچ حالتی با $x < 0$ پیدا نکرد (errors: 0). این نتیجه، مثل خصوصیت Safety، تا حدی بدیهی است: متغیر x از مقدار اولیه ۱۰ شروع می‌شود و تنها در حلقه‌ی اول، دقیقاً ۵ بار یک واحد کاهش می‌یابد؛ بنابراین کمترین مقدار ممکن برای x در این برنامه‌ی خاص برابر ۵ است و ساختاراً هرگز امکان منفی‌شدن وجود ندارد.

۴.۷.۵ سناریوی نقض – برنامه (ب)

دستور اجرا

```
spin -search -ltl nonNegativeX verify.pml
```

```

1      pan: ltl formula nonNegativeX
2      pan:1: assertion violated !((x>=0))) (at depth 44)
3      pan: wrote verify.pml.trail
4
5      (Spin Version 6.5.1 -- 31 July 2020)
6      Warning: Search not completed
7      + Partial Order Reduction
8
9      Full statespace search for:
10     never claim          + (nonNegativeX)
11     assertion violations  + (if within scope of claim)
12     acceptance  cycles   + (fairness disabled)
13     invalid end states   - (disabled by never claim)
14
15     State-vector 36 byte, depth reached 44, errors: 1
16     23 states, stored
17     0 states, matched
18     23 transitions (= stored+matched)
19     0 atomic steps
20     hash conflicts: 0 (resolved)
21
22     pan: elapsed time 0.006 seconds
23

```

تفسیر

در این برنامه x از ۳ شروع می‌شود و در طول ۱۰ تکرار حلقه ($i < 10$) به‌طور پیوسته کاهش می‌یابد. SPIN در عمق ۴۴ یک نقض واقعی پیدا کرد (errors: 1)، یعنی مقدار x در جایی از اجرا منفی شده است. این تأیید می‌کند که فرمول $\square(x \geq 0)$ واقعاً توسط SPIN روی مقادیر پویا بررسی می‌شود، نه اینکه نتیجه‌ی مدل پایه به این دلیل بود که invariant همیشه برقرار است.